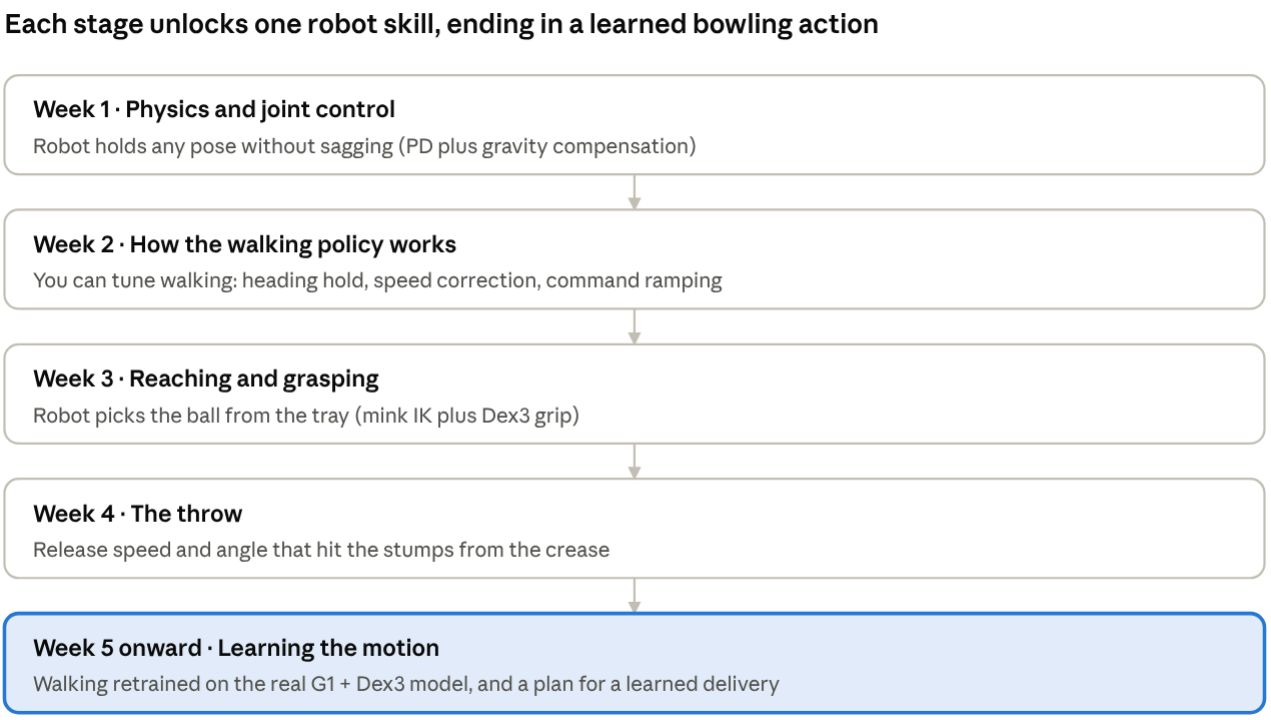
Reading Plan: G1 Bowling Project

Oct 21, 2026 · @vishalNaik

Overview

Five stages take you from "why does the robot sag" to training your own walking policy, each pairing reading with one exercise in the ros_mujoco_g1_bowling repo. Move on when a stage's **Done when** check passes, not when the reading is finished. The paper summaries are in the Reading Guide.



reading plan · 5 stages

Each stage builds on the one above it: grasping needs joint control, and retraining needs to understand the current policy.

Week 1: Physics and joint control

Goal: understand how a target angle becomes a motor torque, and why the robot sagged or shook before the fixes.

Read

- MuJoCo documentation: *Overview* (MjModel vs MjData, the simulation loop) and the Python tutorial notebook linked from the Python page.

- Modern Robotics, ch. 11 *Robot Control*: PD control and gravity compensation (computed torque).

Exercise

1. Open the robot with the base fixed: `ros2 launch g1_dex3_description display.launch.py viewer:=mujoco`. All joints should hold still.
2. In `src/g1_dex3_description/scripts/mujoco_viewer.py`, change the elbow entry in `PD_GAINS` from `40.0` to `5.0`. Relaunch and watch the forearms droop.
3. Run the script with `--fixed-base --no-hold`: every joint goes limp. Compare with step 1.
4. In `make_pd_hold`, comment out the `qfrc_bias` line (gravity compensation) and relaunch with the elbow back at 40. Note the few degrees of sag.
5. Print `data.qpos[7:]` once per second and confirm the joint angles you see match the sag.

Done when you can explain in two sentences why PD alone sags under gravity and why adding `qfrc_bias` fixes it on a fixed base but not on a free one.

Week 2: How the walking policy works

Goal: read `walk.py` and know what every number the policy sees and outputs means, and why the outer loops exist.

Read

- Rudin et al., *Learning to Walk in Minutes Using Massively Parallel Deep RL* (arXiv:2109.11978): the observations, rewards and command setup.
- Schulman et al., *Proximal Policy Optimization Algorithms* (arXiv:1707.06347): the clipped objective, section 3.
- Spinning Up in Deep RL: *Introduction to RL* parts 1–3 and the PPO page, which explains the same paper more gently.

Exercise

1. Open `src/g1_bowling_control/g1_bowling_control/walk.py` and find `_policy_update`. Write down the slice of `obs` for each part: angular velocity (3), gravity direction (3), command (3), joint positions (12), joint velocities (12), last action (12), gait phase (2). They should add up to 47.
2. Find where the action becomes a joint target (`action * action_scale + default`). Explain why the policy outputs offsets from a default pose instead of absolute angles.
3. Run `ros2 launch g1_bowling_control walk.launch.py`, press the up arrow 5 times (0.5 m/s), and note the drift in the printed `pos`.

4. In `config/g1_walk.yaml`, set `heading_kp: 0`, relaunch, repeat step 3, and compare the yaw drift. Restore it afterwards.
5. Set `cmd_accel_limit` to `[5.0, 5.0, 5.0]`, walk at 0.8 m/s and press Home to stop. Watch whether it stumbles, then restore `[0.5, 0.5, 1.0]`.

Done when you can explain the four layers in `walk.py` (command, outer loops, policy at 50 Hz, leg PD every step) and say which line of the config each experiment changed.

Week 3: Reaching and grasping

Goal: move the right Dex3 palm to the ball in the tray and close the fingers on it, with the base fixed.

Read

- Robotic Manipulation (Tedrake): the chapter on basic pick and place (differential inverse kinematics) and the grasping chapter (contact, friction cones, force closure).
- Modern Robotics, ch. 6 *Inverse Kinematics*, for the math behind the Jacobian.
- The examples in the mink GitHub repository (kevinzakka/mink), especially a humanoid one, to see how tasks and limits are set up.

Exercise

1. Create `scripts/reach_ball.py`. Load `bowling_scene.xml` with the base fixed (reuse `load_model` from `g1_bowling_control.sim`).
2. Move the base closer first: the table's front edge is 0.7 m away, so set the pelvis x to about 0.45 m before the loop (or move it in a copy of the scene).
3. With mink, add a frame task that puts `right_hand_palm_link` a few centimetres above the `tray_center` site, plus a posture task to keep the rest of the body near its zero pose. Hold the legs fixed.
4. Each step, solve IK, integrate to new joint targets, and send them through PD (like `JointHold`, with the target updated every step).
5. When the palm is in place, ramp the seven right-hand finger targets from open to closed over about 1 s. Print the contact forces between the finger links and the `ball` body.
6. Lift the palm 10 cm and check that `data.body('ball').xpos[2]` rises with it.

Done when the ball leaves the tray in the hand and stays there for 3 s. If it slips, note whether the cause is grip force (the 1.4 Nm finger limit), friction, or finger placement.

Week 4: The throw

Goal: find the release speed and angle that send the ball from the crease to the stumps, and test it in the scene without the robot.

Read

- Zeng et al., *TossingBot: Learning to Throw Arbitrary Objects with Residual Physics* (arXiv:1903.11239): the projectile model and the learned residual on top of it.
- Any projectile-motion refresher (horizontal and vertical motion under gravity).

Exercise

1. Read the positions you need from the scene: the `release_line` site (crease, y = 7.22 m) and the `target` site (stumps, y = 26.12 m, z = 0.5 m). The horizontal distance d is about 18.9 m.
2. Pick a release height h (where the hand would be at release) and a release angle θ above horizontal. Solve the projectile equation for the release speed v:

```
z_{target} = h + d\tan\theta - \frac{g\,d^2}{2\,v^2\cos^2\theta}
```

3. Write `scripts/test_throw.py`: load `bowling_scene.xml` with the base fixed, place the ball at the crease at height h, set its free-joint velocity to v at angle θ toward the target, and step until the ball stops.
4. Check whether the middle stump or a bail moves. Print the ball's position as it passes y = 26.12.
5. Repeat for three angles. Note how much a 1° angle error or a 0.5 m/s speed error shifts the landing point: this is the accuracy the robot's release must achieve.

Done when a script hits the stumps from the crease, and you have written down the speed and angle tolerances. Ignoring bounce and air drag is fine for now; note them as the residual a learned correction would cover.

Week 5 onward: Learning the motion

Goal: replace the temporary hand-mass workaround with a walking policy trained on the real G1 + Dex3 model, then plan the learned bowling action.

Read

- Tobin et al., *Domain Randomization* (arXiv:1703.06907): why to randomize mass and friction while training.
- Peng et al., *DeepMimic* (arXiv:1804.02717): learning a motion by tracking a reference clip.

- He et al., *ASAP* (arXiv:2502.01143): motion tracking from human video on the Unitree G1.

Exercise

1. Get the GPU working: install the NVIDIA driver, reboot, and confirm `torch.cuda.is_available()` prints `True`.
2. Set up a training framework (mjlab or MuJoCo Playground) with the G1 velocity-tracking task.
3. Swap in your model `g1_29dof_dex3.xml`, randomize hand mass by about ±20% and add a 0.16 kg payload in one hand in some episodes (the ball).
4. Train, export the policy, and run it in `walk.py` with `hand_mass_scale: 1.0`. Compare drift and falls against the current results.
5. For the delivery: find or record a bowling motion clip, retarget it to the G1, and write down the plan for a DeepMimic-style tracking policy.

Done when the retrained policy walks reliably at 0.8 m/s with real Dex3 hand mass, and you have a written plan for the delivery policy.

Progress and notes

Tick a stage off when its done-when check passes.

- [ ] Week 1: physics and joint control
- [ ] Week 2: how the walking policy works
- [ ] Week 3: reaching and grasping
- [ ] Week 4: the throw
- [ ] Week 5 onward: learning the motion

After each paper, fill in this template in a notes file next to your code:

```
## <Paper title> (<year>)
- Problem it solves (1 line):
- Key idea (1 line):
- Where it applies in my repo (file or stage):
- One thing to try:
```